

DEPLOYMENT GUIDE

Deploying 7ALP's on a New Ubuntu Server

Complete, start-to-finish instructions for standing up the frontend and backend on a brand new Ubuntu VPS — written for migrating off the current GCP instance to a different provider, with the real gotchas hit during the first deployment called out inline.

Domain: 7alpsglobal.com · Repo: github.com/Eagleclaw123/7alps (private) · Ubuntu 22.04/24.04 · Generated 2026-09-10

0. Before you touch the new server

Gather these first — you'll need them mid-setup and it's disruptive to stop and go hunting:

- SSH access to the new server (root or sudo user, key-based auth set up)
- Access to whichever DNS panel manages `7alpsglobal.com` (domain registrar, or a separate DNS host if you use one)
- MongoDB Atlas login — you'll add the new server's IP to Network Access
- A copy of the current production secrets: `DATABASE / DATABASE_PASSWORD`, `RESEND_API_KEY`, `JWT_SECRET`, `R2_*` keys, `RAZORPAY_KEY_ID / SECRET` — pull these from the current server's `backend/.env` (see Appendix A for the full list of keys)
- The `VITE_GOOGLE_MAPS_API_KEY` value, and access to the Google Cloud Console project it belongs to (you'll update the allowed HTTP referrers)
- GitHub access to add a new deploy key to `Eagleclaw123/7alps` (repo is private)

Two hard-won lessons from the first deployment (GCP), worth reading before you start:

1. **DNS pointing at the wrong IP looks like a working server.** If an A record points at a stale IP (e.g. a leftover CDN/parking service from the registrar), requests can still get a clean HTTP response — just from the wrong host — making it look like a server-side bug when it's actually DNS. Always verify with `curl -4 ifconfig.me` on the server itself vs. what the domain actually resolves to, before debugging anything else.
2. **Cloud provider firewalls block port 80/443 by default, separately from anything inside the VM.** On GCP this meant nginx was running perfectly and answered fine on `localhost`, yet timed out from the outside world — until "Allow HTTP traffic" / "Allow HTTPS traffic" was enabled at the instance/network level. Every provider has an equivalent (security groups on AWS, cloud firewalls on DigitalOcean/Linode/Hetzner, NSGs on Azure) — check it early, don't assume the OS firewall (`ufw`) is the only gate.

1. DNS — point the domain at the new server

Once the new server has a static public IP, update the A records wherever `7alpsglobal.com`'s DNS is managed:

Type	Host	Value
A	@	<new server's public IP>
A	www	<new server's public IP> (or a CNAME to @)

If the DNS provider offers a "proxy"/"CDN" toggle on the record, leave it **off** (DNS-only) — a proxying layer in front of the origin will interfere with the Let's Encrypt HTTP-01 challenge in step 10 unless specifically configured to pass it through.

Lower the TTL a day ahead of the cutover if possible (e.g. to 300s), so the switch from the old GCP IP to the new server propagates fast when you actually flip it — rather than waiting out a long cached TTL.

Verify propagation before moving on:

```
nslookup 7alpsglobal.com
nslookup www.7alpsglobal.com
# both should resolve to the NEW server's IP
```

2. Open the cloud firewall (before installing anything)

Do this *first*, on whichever provider you've moved to — get it out of the way so it's not a mystery later:

- Find the provider's network/firewall/security-group panel (not something inside the VM)
- Allow inbound TCP **22** (SSH — should already be open, that's how you're connected)
- Allow inbound TCP **80** (HTTP — needed for the Let's Encrypt challenge and for the redirect to HTTPS)
- Allow inbound TCP **443** (HTTPS)

Sanity check from your own machine once the server is reachable (even before nginx is installed, this confirms the network path is open):

```
curl -I http://<new-server-ip> # connection refused is fine (nothing listening yet) – a timeout
means the firewall is still blocking
```

3. Server prep

1 Update the system

```
sudo apt update && sudo apt upgrade -y
```

2 Install Node.js 20 LTS

Vite 8 (used by the frontend) requires Node ≥ 20.19 ; the backend only requires ≥ 16 , so 20 LTS covers both.

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs git nginx
node -v      # confirm v20.x
npm -v
```

3 Install PM2 globally

```
sudo npm install -g pm2
```

4 OS-level firewall (ufw)

Some minimal cloud images (this bit us on the GCP box) don't ship `ufw` pre-installed — install it if missing, then open the same three ports at the OS level too. This is a second, independent gate from the provider firewall in step 2 — you generally need **both** open.

```
sudo apt install -y ufw # skip if already installed
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
sudo ufw status verbose
```

4. Give the server access to the private GitHub repo

```
ssh-keygen -t ed25519 -C "7alpsglobal-deploy" -f ~/.ssh/id_ed25519 -N ""
cat ~/.ssh/id_ed25519.pub
```

Copy the printed public key → GitHub repo `Eagleclaw123/7alps` → **Settings** → **Deploy keys** → **Add deploy key**. Read-only access is sufficient — a server never needs to push.

5. Clone the repo

```
sudo mkdir -p /var/www/7alps
sudo chown -R $USER:$USER /var/www/7alps
git clone git@github.com:Eagleclaw123/7alps.git /var/www/7alps
cd /var/www/7alps
```

6. Backend setup

```
cd /var/www/7alps/backend
npm ci --omit=dev
```

Create `backend/.env` — this is gitignored, so it must be created by hand on every new server. Use Appendix A as the template and fill in the *real* secrets carried over from the old server:

```
nano .env
```

Start it under PM2, using the checked-in `ecosystem.config.js` (Appendix D):

```
mkdir -p logs
pm2 start ecosystem.config.js --env production
pm2 save
pm2 logs 7alps-backend --lines 50 # confirm "DB connection successful!" and "App running on port 3000..."
```

Make PM2 survive reboots:

```
pm2 startup systemd
# it prints a sudo command – copy/paste and run EXACTLY what it outputs
pm2 save
```

7. MongoDB Atlas — allow the new server's IP

In Atlas → your project → **Network Access**, add the new server's public IP to the allowlist. The backend will fail every database call (visible in `pm2 logs`) until this is done. Once the new server is confirmed fully working end-to-end, remove the old GCP server's IP from the allowlist.

8. Frontend build

```
cd /var/www/7alps/frontend
npm ci
```

Create `frontend/.env.production` from Appendix B — Vite bakes these values into the JS bundle at build time, so this file must exist *before* every build, and changing it later requires a rebuild, not a restart:

```
nano .env.production
```

```
npm run build # produces frontend/dist – this is what nginx serves
```

9. nginx

Install the config from the repo (Appendix C) — it already assumes the `/var/www/7alps` path used above:

```
sudo cp /var/www/7alps/deploy/nginx.conf /etc/nginx/sites-available/7alpsglobal.com
sudo ln -s /etc/nginx/sites-available/7alpsglobal.com /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

At this point `http://7alpsglobal.com` should already load the frontend, and `http://7alpsglobal.com/api/v1/...` should hit the backend. Verify before moving to SSL:

```
curl -I http://7alpsglobal.com
curl http://7alpsglobal.com/api/v1/products/public
```

10. HTTPS (Let's Encrypt)

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d 7alpsglobal.com -d www.7alpsglobal.com
```

Certbot edits `/etc/nginx/sites-available/7alpsglobal.com` in place, adding the 443/SSL server block and (if you accept the prompt) an HTTP→HTTPS redirect. Verify auto-renewal is wired up:

```
sudo systemctl status certbot.timer
sudo certbot renew --dry-run
```

If this fails with a challenge error, before anything else confirm: (a) DNS actually resolves to *this* server — `nslookup 7alpsglobal.com` vs. `curl -4 ifconfig.me` on the server, and (b) port 80 is reachable from the outside — `curl -I http://7alpsglobal.com` from your own machine, not from the server itself. Both of these bit us on the first deployment; see the callout in Section 0.

11. Google Maps API key

In Google Cloud Console, update the `VITE_GOOGLE_MAPS_API_KEY`'s HTTP referrer restriction to include the new domain (it's the same domain, so this is likely already correct — just confirm it wasn't accidentally scoped to an old IP or staging URL):

```
https://7alpsglobal.com/*
https://www.7alpsglobal.com/*
```

12. Full smoke test

- `curl -I https://7alpsglobal.com` — 200 over HTTPS
- `curl https://7alpsglobal.com/api/v1/products/public` — real JSON, not an error
- In a real browser: customer signup (email OTP arrives via Resend), login, add to cart, checkout with both COD and Razorpay test payment
- Admin login, load the dashboard, confirm real stats appear
- B2B login and a quote request, including the Google Maps address picker
- Upload a product image in the admin panel — confirms R2 credentials are correct on the new server

13. Cutover & decommission

1. Confirm the smoke test above is fully clean on the new server *while the old GCP server is still up* — don't decommission anything yet.
2. Flip DNS (if not already done in Section 1) and wait for propagation.
3. Watch traffic/logs on the new server for a short burn-in period.
4. Remove the old GCP server's IP from MongoDB Atlas's Network Access list.
5. Only once confident: stop the old GCP instance, then delete it (and its reserved static IP, if billed separately) to stop paying for both.

Appendix A — backend/.env template

deploy/backend.env.example

```
# Copy to backend/.env on the server and fill in real values.
# NEVER commit this file with real secrets – backend/.gitignore already ignores .env

NODE_ENV=production
PORT=3000

DATABASE=<your MongoDB Atlas SRV string with <PASSWORD> placeholder kept literally>
DATABASE_PASSWORD=<atlas password>

RESEND_API_KEY=<resend api key>
RESEND_FROM_EMAIL=<verified sender e.g. 7alps@lanvaus.com>

JWT_SECRET=<long random string>
JWT_EXPIRES_IN=90d
JWT_COOKIE_EXPIRES_IN=90

R2_ACCESS_KEY_ID=<cloudflare r2 key>
R2_SECRET_ACCESS_KEY=<cloudflare r2 secret>
R2_ACCOUNT_ID=<cloudflare r2 account id>
R2_BUCKET_NAME=<bucket name>
R2_PUBLIC_URL=<public bucket url>

RAZORPAY_KEY_ID=<live or test key id>
RAZORPAY_KEY_SECRET=<live or test key secret>

# Comma-separated list of origins allowed to call the API with credentials.
# Same-origin requests through the nginx proxy don't need this, but keep it
# set in case anything ever calls the API cross-origin.
CORS_ORIGINS=https://7alpsglobal.com,https://www.7alpsglobal.com
```

Appendix B — frontend/.env.production template

deploy/frontend.env.production.example

```
# Copy to frontend/.env.production on the server before running `npm run build`.
# Vite bakes these values into the JS bundle at build time – there is no
# runtime env for a static build, so this file must exist before every build.

VITE_API_BASE_URL=https://7alpsglobal.com/api/v1
VITE_APP_URL=https://7alpsglobal.com
VITE_GOOGLE_MAPS_API_KEY=<same key as production Google Cloud Console project>
```

Appendix C — nginx config

deploy/nginx.conf

```

server {
    listen 80;
    listen [::]:80;
    server_name 7alpsglobal.com www.7alpsglobal.com;

    root /var/www/7alps/frontend/dist;
    index index.html;

    client_max_body_size 10M;

    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml
application/xml+rss text/javascript image/svg+xml;
    gzip_min_length 1024;

    location /api/ {
        proxy_pass http://127.0.0.1:3000/api;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    location /images/ {
        proxy_pass http://127.0.0.1:3000/images;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /assets/ {
        try_files $uri =404;
        access_log off;
        add_header Cache-Control "public, max-age=31536000, immutable";
    }

    location / {
        try_files $uri $uri/ /index.html;
        add_header Cache-Control "no-cache";
    }
}

```

Appendix D — PM2 ecosystem file

backend/ecosystem.config.js

```
module.exports = {
  apps: [
    {
      name: '7alps-backend',
      script: 'server.js',
      cwd: __dirname,
      instances: 1,
      exec_mode: 'fork',
      env_production: {
        NODE_ENV: 'production',
      },
      max_memory_restart: '300M',
      out_file: './logs/out.log',
      error_file: './logs/error.log',
      merge_logs: true,
      time: true,
    },
  ],
};
```

7ALP's — Fresh Server Setup Guide. Companion to the Future Deployments guide, which covers day-to-day updates once this initial setup is done.